

Юнит-тестирование

юнит-тест

фреймворки юнит-тестов

правила именования

преимущества написания юнит-тестов

кто и когда должен писать юнит-тесты

модель AAA (Arrange, Act, Assert)

ограничения Assert

параметризованные тесты

- *Модульное тестирование* максимально влияет на качество кода, когда оно является неотъемлемой частью рабочего процесса разработки ПО.
- После кода создаются модульные тесты, которые
 - проверяют поведение кода в ответ на стандартные, граничные и некорректные случаи ввода данных;
 - также проверяются любые явные или предполагаемые допущения, сделанные кодом.
- При *разработке, управляемой тестами*, модульные тесты создаются перед написанием кода, поэтому модульные тесты используются в качестве технической документации и спецификации функциональности.

- **Юнит-тест** — это функция, которая вызывает другую функцию (юнит) и затем проверяет правильность конечного результата вызванной функции.
- если предположение о конечном результате не подтверждается, то считается, что юнит тест завершился не удачно
- если предположение подтверждается, то юнит-тест выполняется
- тестируется
 - отдельный метод класса
 - или часть кода метода.

Особенности.

- Упор на небольшие участки позволяет довольно быстро разрабатывать простые тесты.
- Однажды написанный код нередко читают многократно, поэтому важно писать понятный код.
- При тестировании разработчик должен быстро прочесть исходный код и понять в чем проблема и как ее исправить.

Характеристики хорошего модульного теста

- **Быстрый.** В хорошо разработанных проектах могут быть тысячи модульных тестов. Модульные тесты должны выполняться очень быстро. За миллисекунды.
- **Изолированный.** Модульные тесты являются автономными, могут выполняться изолированно и не имеют зависимостей от внешних факторов, таких как файловая система или база данных.
- **Повторяемый.** Запуски модульного теста должны иметь согласованные результаты, то есть всегда возвращать одинаковый результат, если вы не вносите никаких изменений между запусками.
- **Самопроверяющий.** Тест должен автоматически определять, пройден он или нет, без участия пользователя.
- **Уместный.** Время на написание модульного теста не должно значительно превышать время написания тестируемого кода. Если вам кажется, что тестирование кода занимает слишком много времени по сравнению с написанием кода, продумайте структуру, более подходящую для тестирования.

- **Тестируемая система (SUT, system under test)** – класс, содержащий единицы (юниты) тестирования.
- **Интеграционное тестирование** – это тестирование единиц работы при отсутствии полного контроля над ней, с использованием одной или нескольких реальных зависимостей, например от времени, сети, данных, потоков, генератора случайных чисел и т.д.
- **Разница интеграционного и юнит-тестирования**
 - Юнит-тесты изолируют систему от зависимостей
 - Позволяет четко определить возможность ошибок именно в этом изолированном коде.
 - Упрощает и повышает контроль над отдельными компонентами программы.
 - Юнит тесты должны быть более быстрыми и стабильными

Фреймворки для юнит-тестов

- **xUnit.net:** фреймворк тестирования для платформы .NET. Наиболее популярный фреймворк для работы именно с .NET Core и ASP.NET Core. НО плохо документирован.
- **MS Test:** фреймворк юнит-тестирования от компании Microsoft, который по умолчанию включен в Visual Studio и который также можно использовать с .NET Core
MS Test: содержит не все необходимые функции для тестирования (например отсутствуют параметризованные тесты)
- **NUnit:** портированный фреймворк с JUnit для платформы .NET
NUnit: один из старейших и функциональных фреймворков тестирования .NET
- Фреймворки юнит-тестирования слабо отличаются друг от друга
- При понимании написания юнит-теста на одном фреймворке легко перейти на другой

Начало работы с юнит-тестами. Демонстрация

- <https://docs.microsoft.com/ru-ru/visualstudio/test/getting-started-with-unit-testing?view=vs-2019>
- <https://docs.microsoft.com/ru-ru/visualstudio/test/unit-test-basics?view=vs-2019#the-mybank-solution-example>

Правила именования

- Проектам присваиваются подобные имена: [ПроектТестирования].Tests
- Классы называются: [ИмяТестируемогоКласса]Tests

Имя теста должно состоять из трех частей:

- имя тестируемого метода;
- сценарий, в котором выполняется тестирование;
- ожидаемое поведение при вызове сценария.

Плохо:

```
public void Test_Single()
```

Лучше:

```
public void Add_SingleNumber_ReturnsSameNumber()
```

Правила именования

- **Способы именования методов:**
- Имя метода юнит-теста начинается с Should, а далее идет описание метода тестирования, например `void ShouldAddTwoNumbers`
- Имя метода юнит-теста начинается с тестируемой программной единицы, далее идет подчеркивание Should и описание метода тестирования, например `void Sum_ShouldAddTwoNumbers`
- Имя программной единицы, которую мы тестируем, подчеркивание, во второй части сценарий тестирования (может быть просто аргумент), подчеркивание, ожидаемое поведение (`ReturnsValue`, `ChangesState`, `CallsDependency`)

Модель AAA (Arrange, Act, Assert)

- Подраздел **Размещение** (Arrange) метода модульного тестирования инициализирует объекты и устанавливает значение данных, которые переданы методу для теста.
- Подраздел **Действие** (Act) вызывает метод для теста с размещенными параметрами.
- Подраздел **Утверждение** (Assert) проверяет, чтобы метод для теста действовал, как ожидается.

Модель AAA (Arrange, Act, Assert)

Плохо:

```
public void Add_EmptyString_ReturnsZero()
{
    // Arrange
    var stringCalculator = new StringCalculator();

    // Assert
    Assert.Equal(0, stringCalculator.Add(""));
}
```

Лучше:

```
public void Add_EmptyString_ReturnsZero()
{
    // Arrange
    var stringCalculator = new StringCalculator();

    // Act
    var actual = stringCalculator.Add("");

    // Assert
    Assert.Equal(0, actual);
}
```

Преимущества применения юнит-тестирования

- можно проверить корректность работы отдельных методов без запуска приложения;
- можно отследить «плохие» изменения в проекте;
- добавляется документация, которая описывает правильное поведение системы;
- большая гибкость в разработке сложных систем;
- меньше дефектов;
- снижение денежных затрат;
- юнит-тесты быстрее ручного тестирования.

Кто и когда должен писать юнит-тесты

- юнит-тесты пишет разработчик;
- юнит-тесты создают подушку безопасности для разработчика;
- юнит-тесты пишутся для любых сложных приложений;
- тестировщики отвечают за ручные тесты.

Утверждения (Assert)

- модель утверждений – основа любого фреймворка юнит-тестирования;
- современный NUnit реализует текучую модель утверждений на основе ограничений;
- `Assert.That<T>(T actual, IResolveConstraint expression).`

Логические ограничения

Второй уровень
логических
ограничений:

- All
- Not
- Some

Верхний уровень
логических
ограничений:

- Is
- Has
- Does

Ограничение All

```
string[] array = new srtring[] {"abc","abc","abc"};  
Assert.That(array, Is.All.Contains("b"));
```

```
int[] array = new int[] { 1, 2, 3, 4, 5 };  
Assert.That(array, Has.All.GreaterThan(0));
```

Ограничение All вызывается на ограничение Is и Has.

Второй уровень
логических
ограничений:

All
Not
Some

Верхний уровень
логических
ограничений:

Is
Has
Does



Ограничение Not

```
Assert.That(array, Is.Not.Length.EqualTo(4));
```

```
Assert.That(@"C:\tmp.txt", Does.Not.Exist);
```

```
Assert.That(42, Is.Not.Null);
```

```
Assert.That(42, Is.Not.True);
```

```
Assert.That(42, Is.Not.False);
```

```
Assert.That(2.5, Is.Not.NaN);
```

```
Assert.That(2 + 2, Is.Not.EqualTo(3));
```

Ограничение Not позволяет создавать отрицательные логические выражения

Ограничение Some

```
object[] strings = new object { "abc", "bad", "bad", "dad" };  
Assert.That(strings, Has.Some.StartsWith("ba"));
```

```
object[] doubles = new object { 0.99, 2.1, 3.0, 4.05 };  
Assert.That(doubles, Has.Some.EqualTo(1.0).Within(.05));
```

Утверждение Has чаще всего используется с ограничением Some

Второй уровень
логических
ограничений:

All
Not
Some

Верхний уровень
логических
ограничений:

Is
Has
Does



Ограничение Does

```
string phrase = "Are you OK?";  
Assert.That(phrase, Does.EndsWith("!"));  
Assert.That(phrase, Does.Not.EndsWith("?"));  
Assert.That(phrase,  
Does.Not.Contain("goodbye"));
```

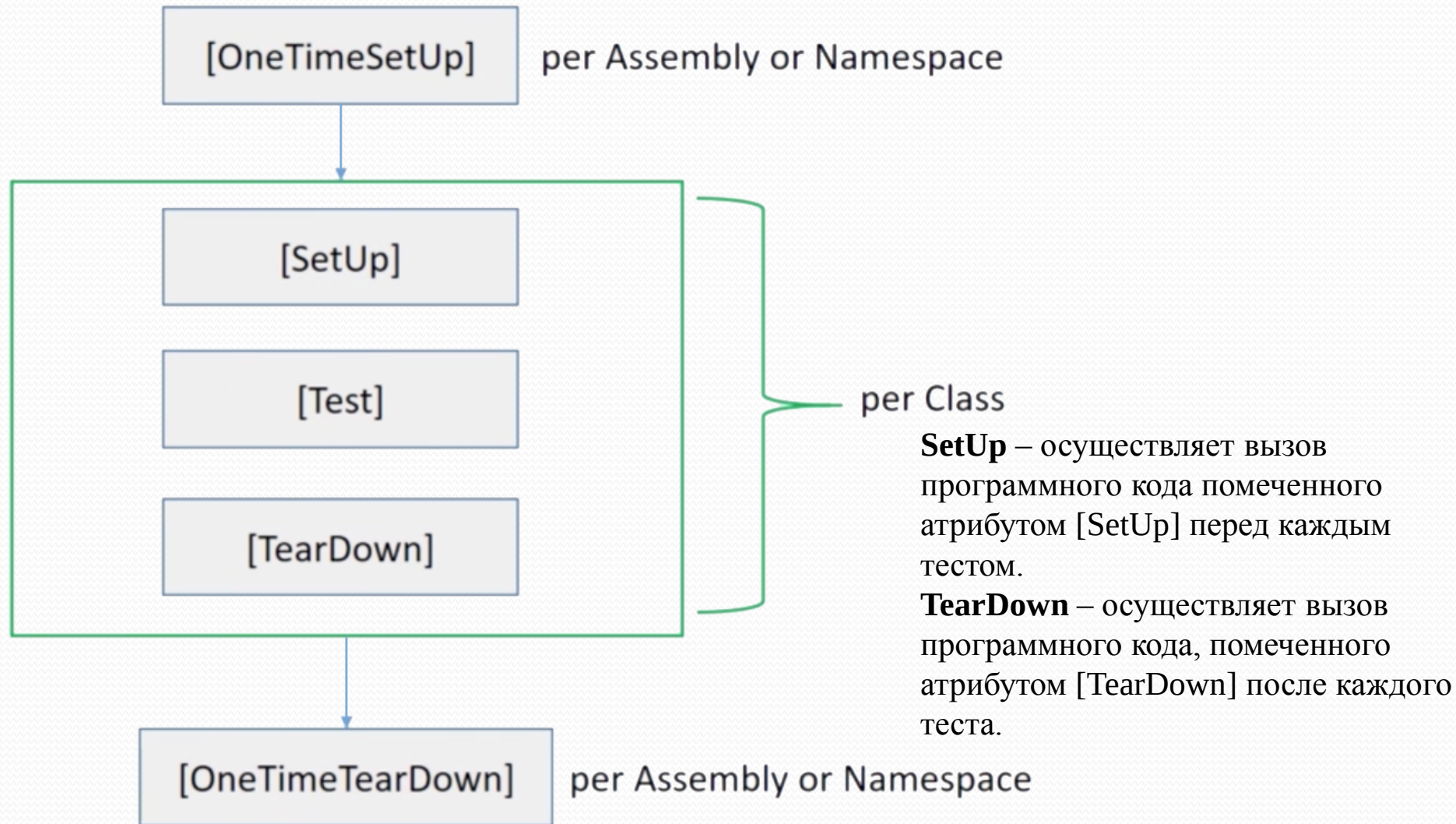
Ограничение Does чаще всего используется на ограничениях, которые пишутся в отношении строк.

Комбинация ограничений

```
Assert.That(5, Is.LessThan(1).Or.GreaterThan(10));
```

```
Assert.That(5, Is.LessThan(10).And.GreaterThan(1));
```

Атрибуты SetUp и TearDown



Не параметризованные тесты

```
[Test]
public void Health_Damage100OnElf_Returns45()
{
    _character.Damage(100);
    Assert.That(_character.Health, Is.EqualTo(45));
}
```

```
[Test]
public void Health_Damage80OnElf_Returns65()
{
    _character.Damage(80);
    Assert.That(_character.Health, Is.EqualTo(65));
}
```

НОВЫЕ значения передаются в НОВОМ тесте

Параметризованные тесты

В параметризованных тестах значения передаются через [TestCase()]

```
[TestCase(100, 45)]  
[TestCase(80, 65)]  
public void Health_Damage_ReturnsCorrectValue(int damage, int expectedHealth)  
{  
    _character.Damage(damage);  
    Assert.That(_character.Health, Is.EqualTo(expectedHealth));  
}
```


Группировка и игнорирование юнит-тестов

Группировка производится с помощью конструкции
`[Category("NameOfCategory")]`

Группировка позволяет запускать только необходимые тесты
(для запуска используется вкладка группировка по
признакам)

Игнорирование производится с помощью конструкции
`[Ignore("Reason")]`

Покрывтие кода

- Позволяет определить, какая часть кода проекта в действительности тестируется закодированными тестами, такими как модульные тесты.
- Для обеспечения эффективной защиты от ошибок тесты должны выполнять ("покрывать") большую часть кода.
- Задание чрезмерно большого процента объема протестированного кода может снизить производительность.
- Не является индикатором успеха

Рекомендации

- Правильно именуйте тесты
- Используйте AAA
- Пишите минималистичные тесты (входные данные должны быть как можно проще)
- Избегайте "магических" строк
- Избегайте логики в тестах
- Выбирайте вспомогательные методы вместо установки и удаления
- Избегайте нескольких проверочных утверждений
- Проверяйте закрытые методы путем модульного тестирования открытых методов
- Используйте заглушки для статических ссылок

Что можно исправить?

[Fact]

```
public void Add_EdgeCases_ThrowsArgumentException()  
{  
    Assert.Throws<ArgumentException>(() => stringCalculator.Add(null));  
    Assert.Throws<ArgumentException>(() => stringCalculator.Add("a"));  
}
```

[Theory]

[InlineData(null)]

[InlineData("a")]

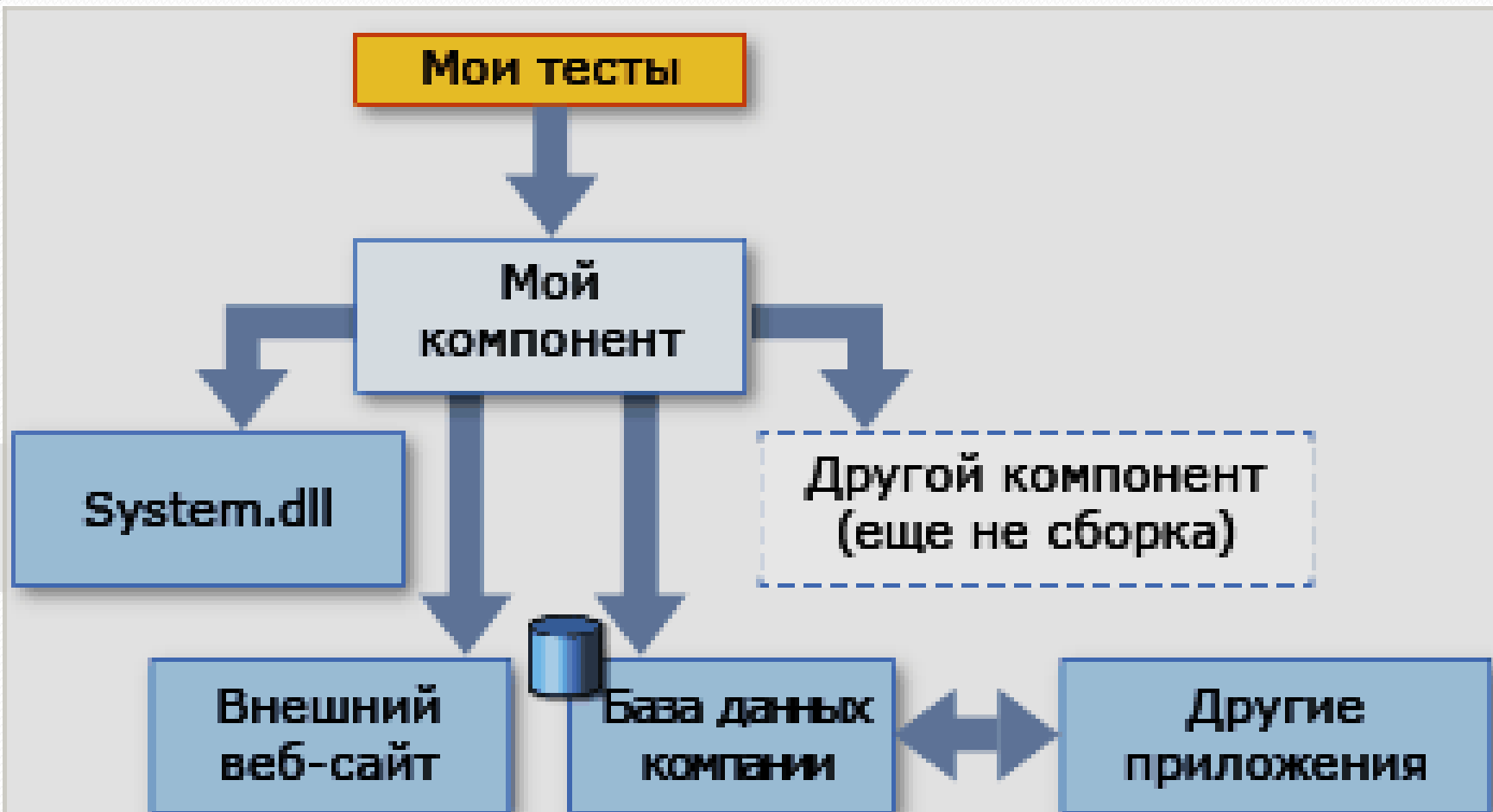
```
public void Add_InputNullOrAlphabetic_ThrowsArgumentException(string  
input)  
{  
    var stringCalculator = new StringCalculator();  
  
    Action actual = () => stringCalculator.Add(input);  
  
    Assert.Throws<ArgumentException>(actual);  
}
```

Какой метод будем тестировать?

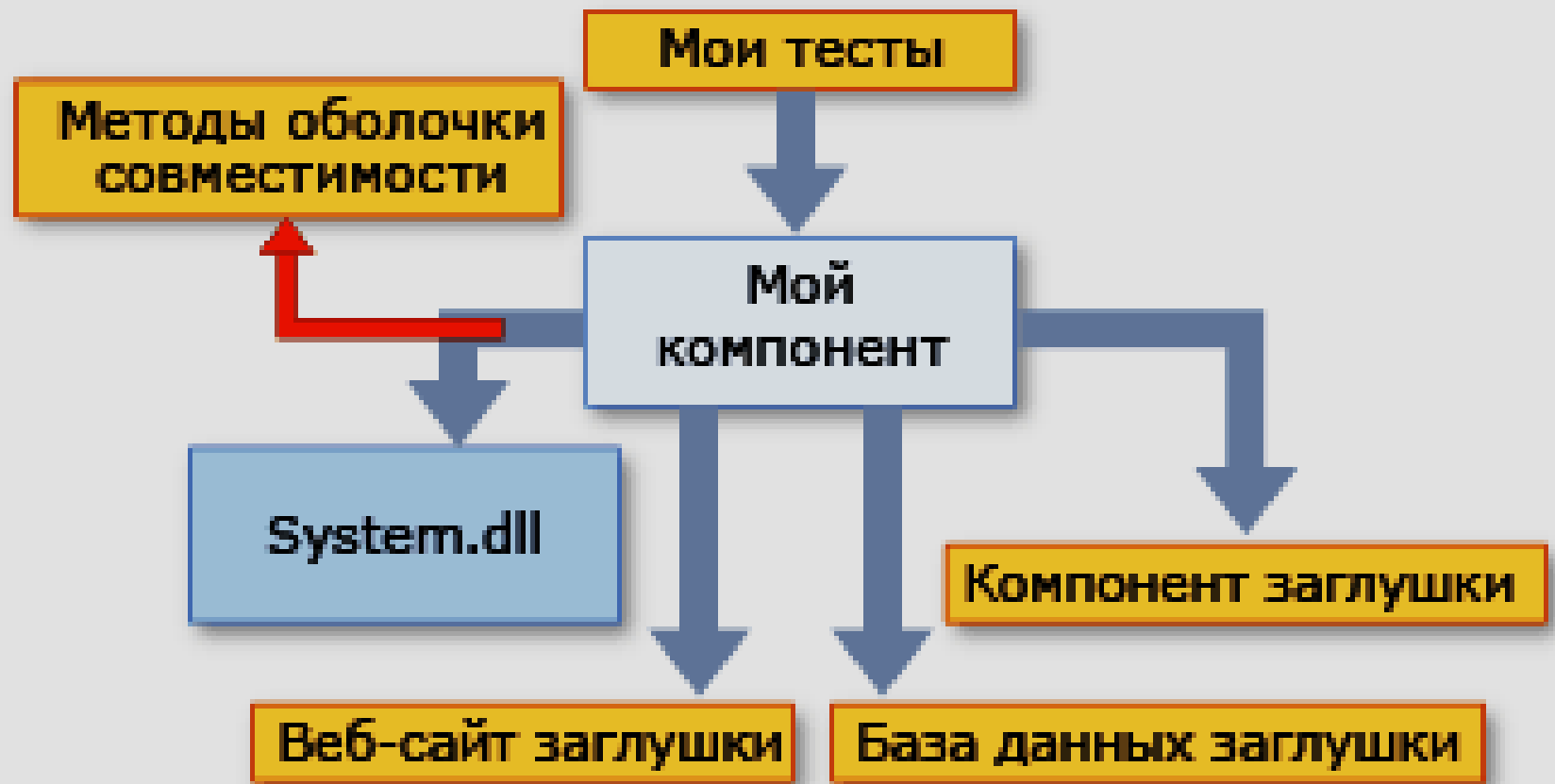
```
public string ParseLogLine(string input)
{
    var sanitizedInput = TrimInput(input);
    return sanitizedInput;
}
```

```
private string TrimInput(string input)
{
    return input.Trim();
}
```

Изоляция кода



Развернутый компонент зависит от внешних частей, которые являются неполными или демонстрируют переменчивое поведение.

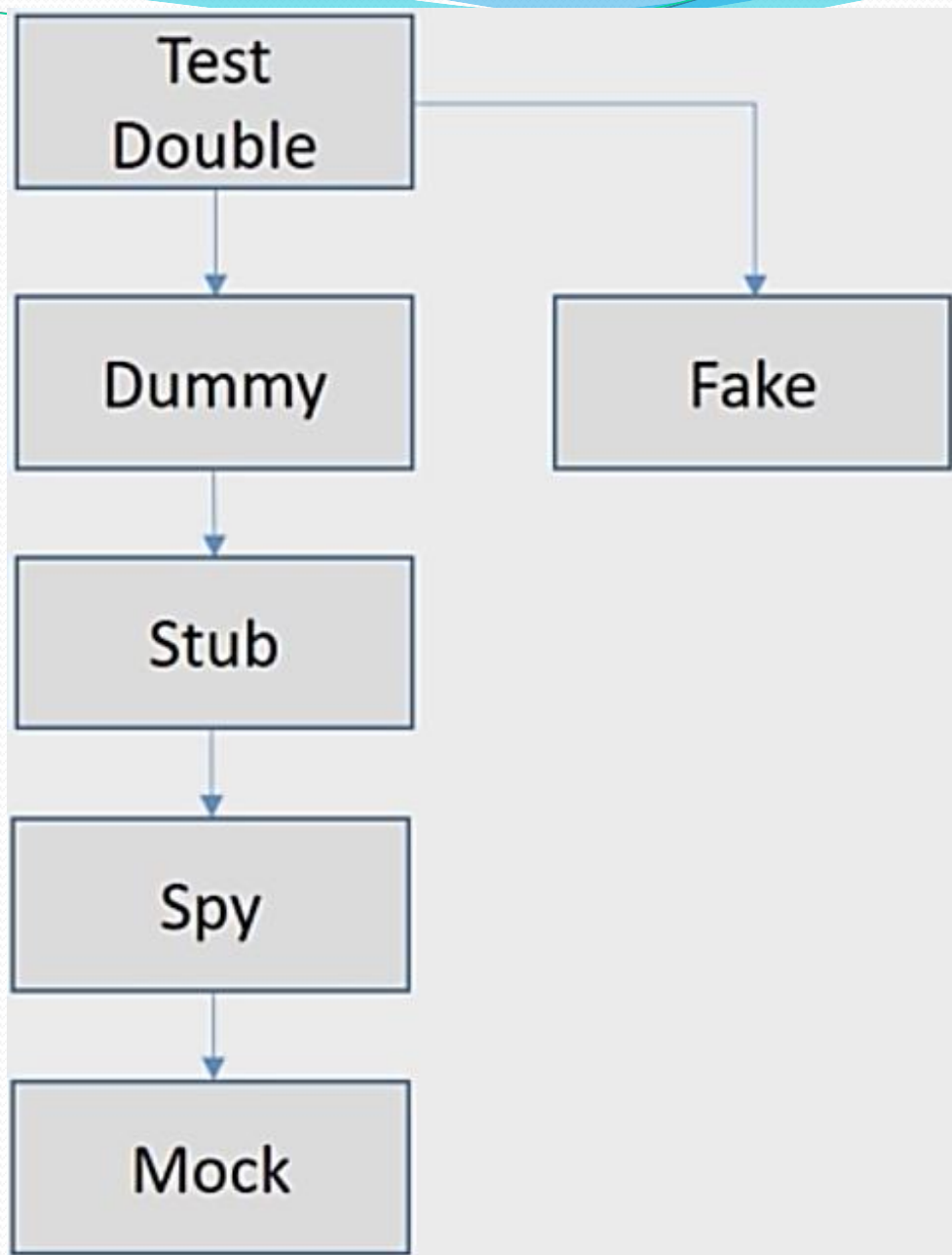


Компонент, изолированный для тестирования. Заглушки и оболочки совместимости работают под контролем тестов.

Виды тестовых дублеров

Для контроля зависимостей используются дублеры, которые имитируют поведение этих зависимостей

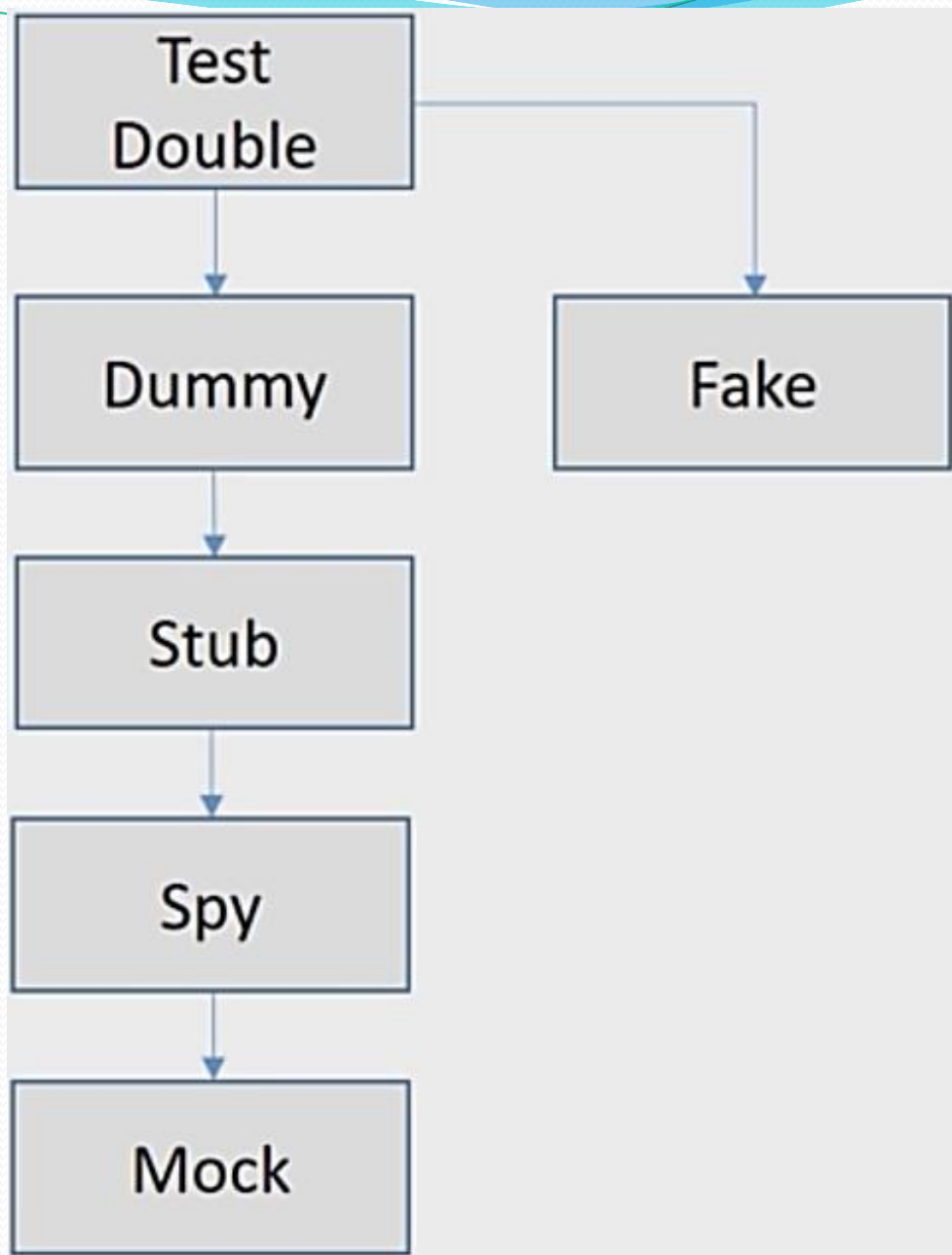
Дублер это общий термин для обозначения любых специальных объектов тестирования, заменяющих реальные зависимости



Виды тестовых дублеров

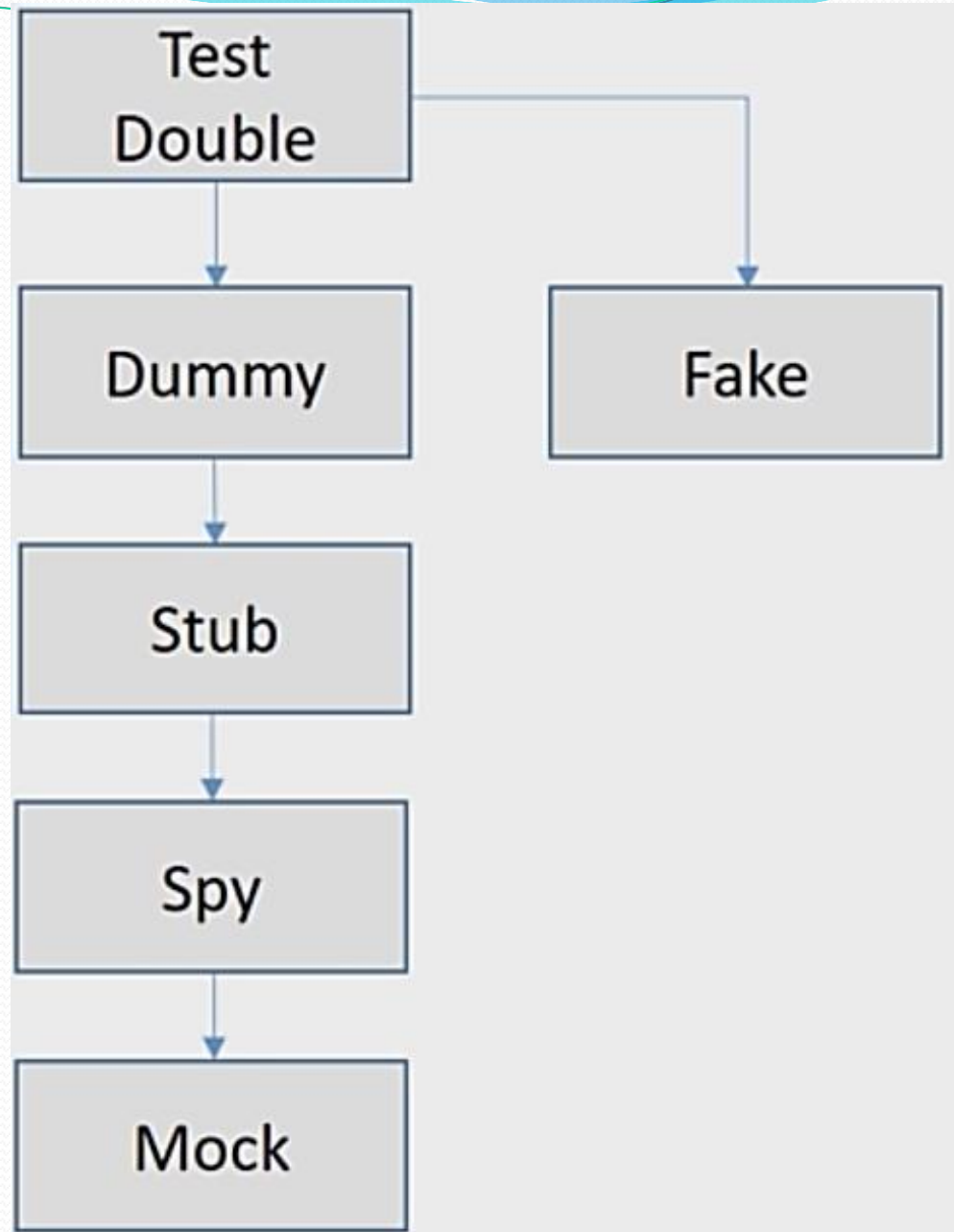
Пустышка (dummies) – дублер возвращающий значения, которые или равны null, в случае возвращения ссылочного типа или близки к 0 в случае структур

В методах где пустышки ничего не должны возвращать, они ничего не делают



Виды тестовых дублеров

Заглушка (stubs) –
пустышка которая
возвращает
предопределенное
постоянное значение из
методов вызываемых
подмененным объектом



Виды тестовых дублеров

Заглушки для управления потоком выполнения через определенные пути.

Пустышка и заглушка проверяют проверяют правильность **возвращаемых значений** и **корректность изменения состояний**.

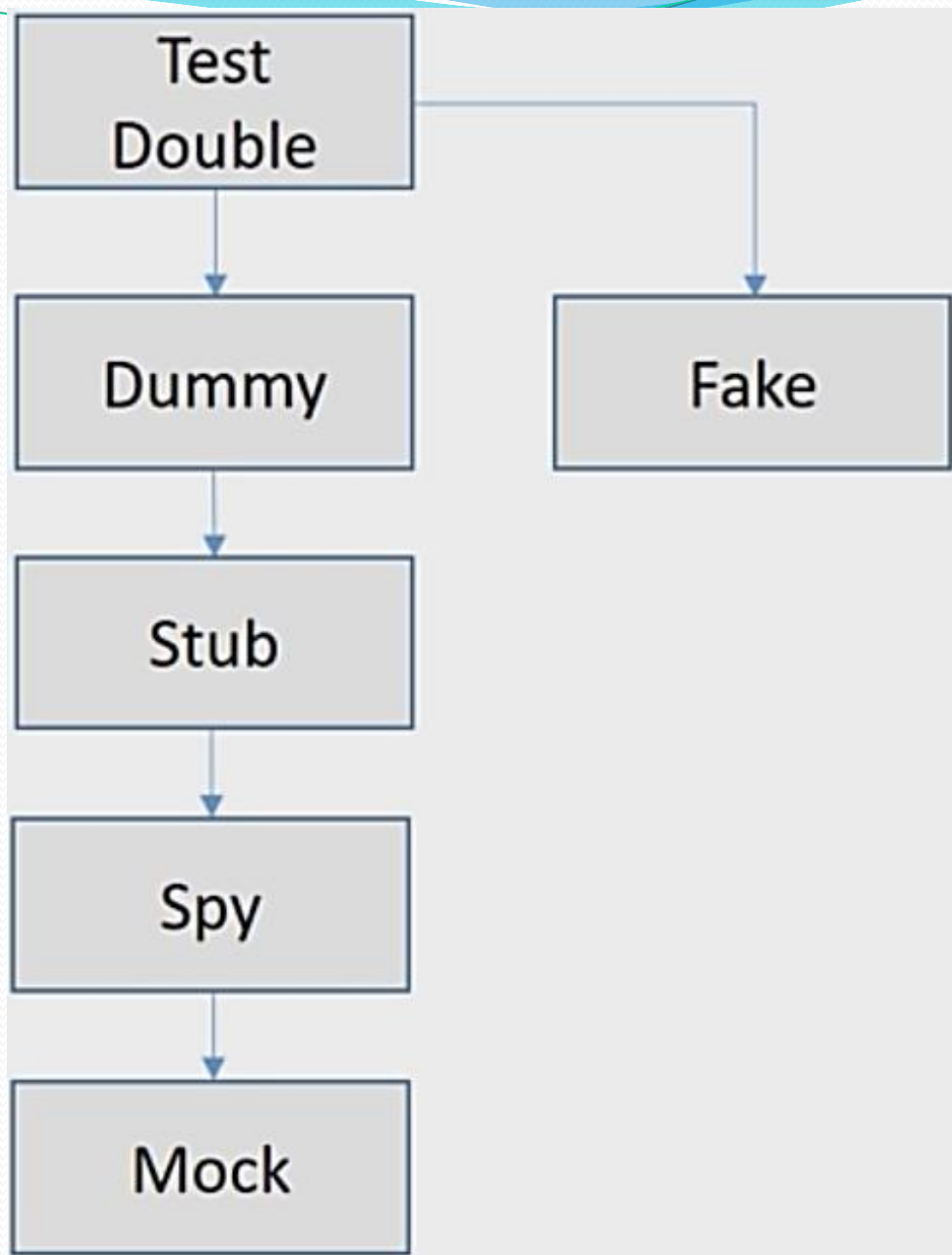
Тестирование взаимодействий – это тестирование того как объект отправляет сообщение или же вызывает методы другим объектам. Используется когда **вызов метода** зависимого объекта является **результатом работы тестируемого объекта**.

Виды тестовых дублеров

Шпион (spies) – заглушка которая хранит часть информации о вызовах сделанных через этот самый шпионский объект.

Позволяет проверять в тесте какие значения аргументов были переданы в методы.

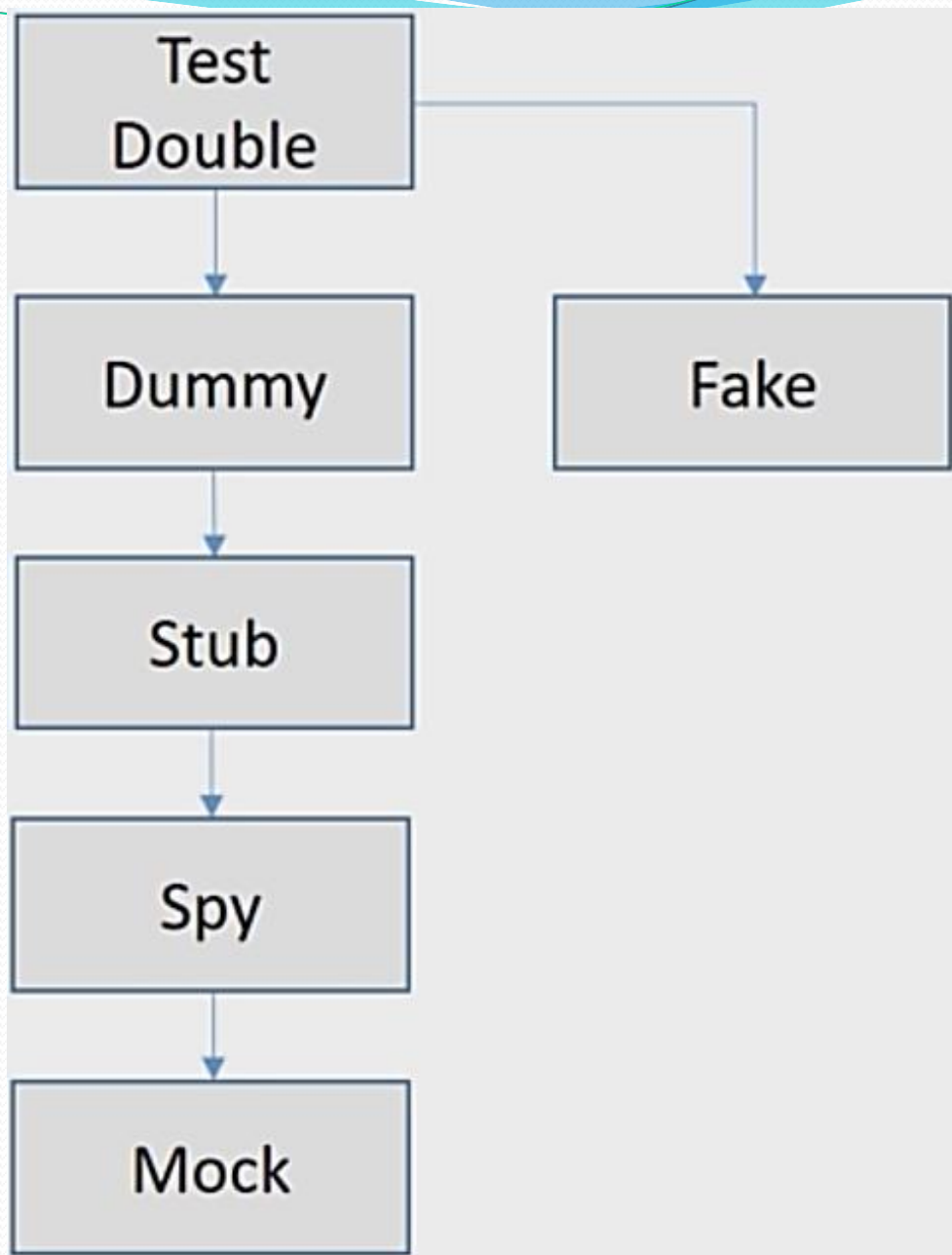
Сколько раз вызывались методы, в каком порядке были вызваны и т.д.



Виды тестовых дублеров

Подставка (mocks) – шпион однако она содержит утверждения внутри себя, значит может прервать работу теста.

Подставка хранит некоторую информацию и если в прогоне теста что-то явно пойдет не так, то ее внутренней утверждение провалится. Подставка знает что должно произойти в случае тех или иных



Виды тестовых дублеров

Подделка (fakes) – дублер, который имитирует поведение настоящей зависимости, как можно ближе к реальности. Подделки как правило не используются из-за сложности обновления, при обновлении зависимости.

